7CS034 Data Science

Week 5 Quarto

Dr. Liam Naughton

School of Mathematics & Computer Science
UoW

Introduction

- This week we'll learn about how we can use `R` markdown and `Quarto` to produce technical documents for Data Science & AI.
- In the lecture we'll spend a significant portion of time looking at some of the features and functionality of `Quarto`.
- In the workshops we'll work through an example where we create our own `Quarto` report based on a dataset which aligns with the requirements which you have been given for your assessment.

What is Quarto?

- An open-source scientific and technical publishing system.
- You can use it with Jupyter notebooks or with plain text markdown in your favorite editor.
- It's designed to work with Python, `R`, Julia and Observable.
- We'll only use it with `R` via `RStudio` but do feel free to browse the functionality with Observable yourselves.
- `Quarto` is used to publish reproducible, production quality articles, presentations, websites, blogs, and books in HTML, PDF, MS Word, ePub, and more.
- It's based on Pandoc which means that it can be used to work with many, many different markup/markdown systems - think of this to mean "human readable".

Markup vs. Markdown

- A markup language is “a notation used to annotate a document’s content to give information regarding the structure of the text or instructions for how it is to be displayed”.
- “A markup language is a text-encoding system consisting of a set of symbols inserted in a text document to control its structure, formatting, or the relationship between its parts. Markup is often used to control the display of the document or to enrich its content to facilitate automated processing. ”
- Examples include TeX, LaTeX, HTML, XML.
- Markdown is one such language - `R` Markdown is a variant which has been designed for `R`.

Markup vs. Markdown

https://en.wikipedia.org/wiki/Markdown

R Markdown

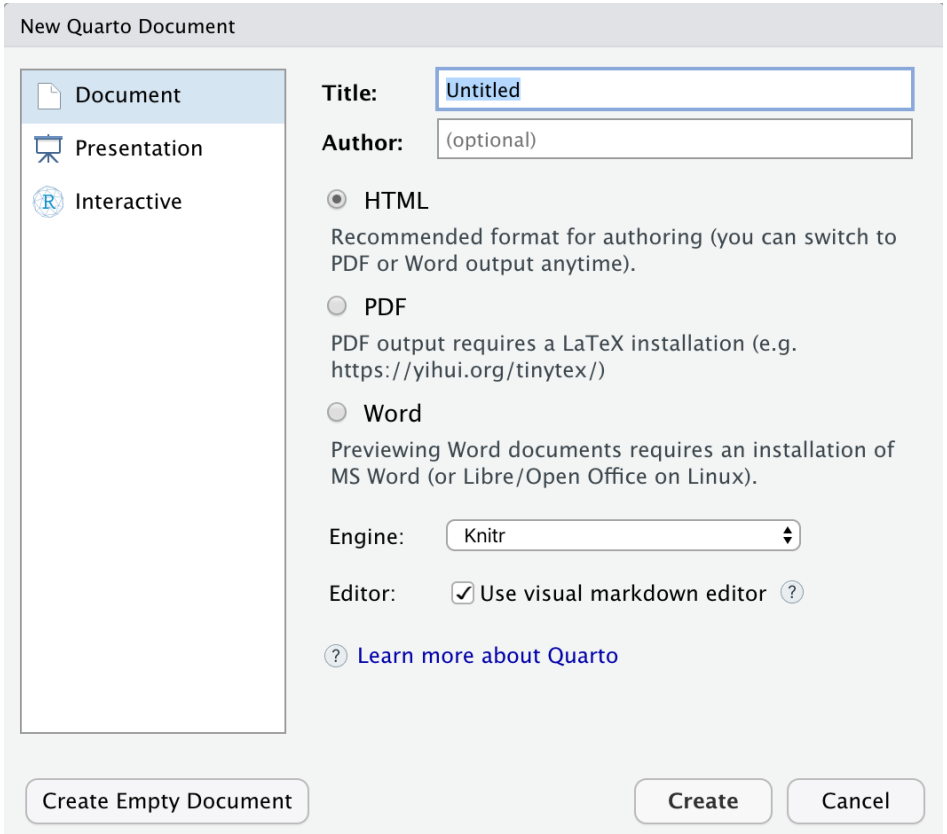
https://rmarkdown.rstudio.com/gallery.html

Using Quarto

- RStudio v2022.07.1 and later includes support for editing and preview of Quarto documents.
- If you are using Quarto within RStudio, it is strongly recommended that you use RStudio v2022.07.1 or a later version.
- A stable release of Quarto is bundled with RStudio v2022.07.1, and later. Upgrading to new versions of RStudio in the future will also upgrade the bundled Quarto version.
- The version of `Rstudio/R` included in AppsAnywhere includes Quarto.

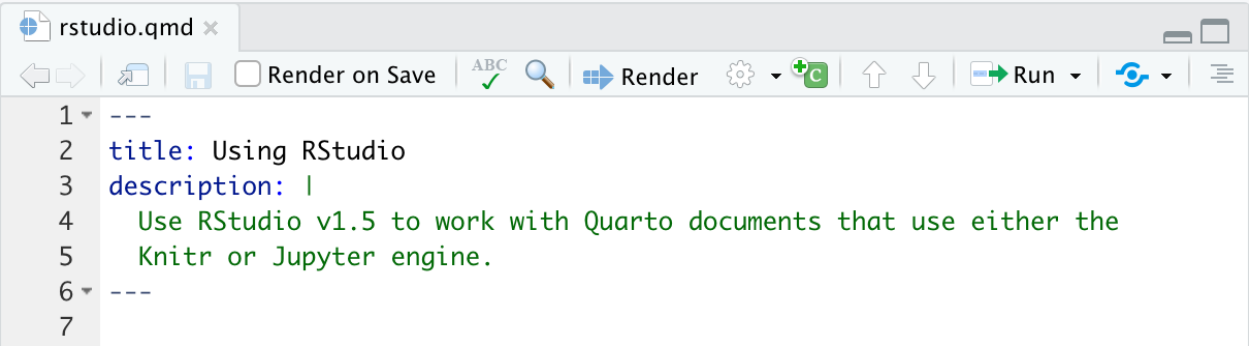
Creating a Quarto Document

- Use the File > New File > Quarto Document command to create new Quarto documents, which will launch the New Quarto Document wizard:

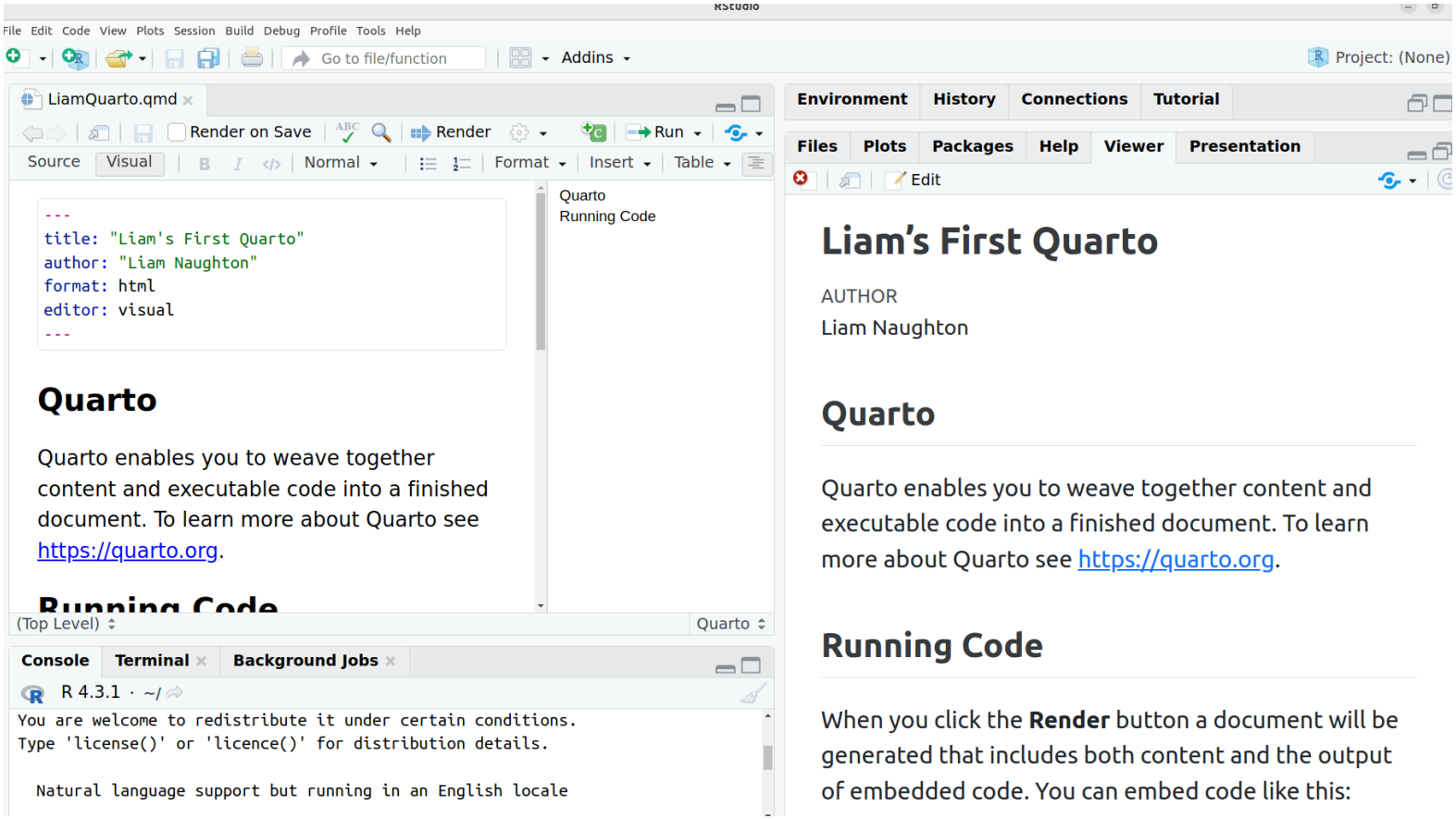


Creating a Quarto Document

- Use the Render button to preview documents as you edit them:



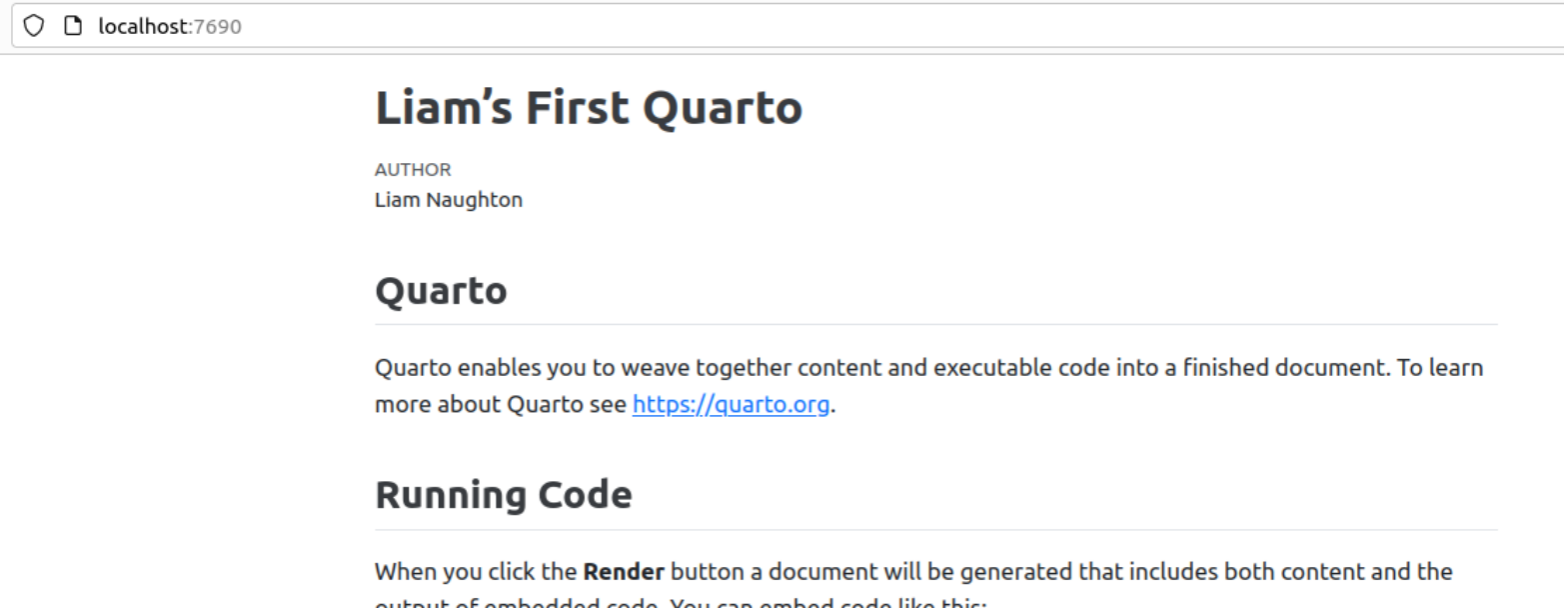
- If you prefer to automatically render whenever you save you can check the Render on Save option on the editor toolbar.
- The preview appears alongside the editor in the Viewer pane if Preview in Viewer Pane is selected:


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function
Addins
Project: (None)
LiamQuarto.qmd
Render on Save
Render
Run
Source
Visual
Normal
Format
Insert
Table
---
title: "Liam's First Quarto"
author: "Liam Naughton"
format: html
editor: visual

Quarto
Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see https://quarto.org.
Running Code
Quarto
Running Code
(Top Level)
Quarto
Console
Terminal
Background Jobs
R 4.3.1 · ~/
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.
Natural language support but running in an English locale
Environment
History
Connections
Tutorial
Files
Plots
Packages
Help
Viewer
Presentation
Edit
Liam's First Quarto
AUTHOR
Liam Naughton
Quarto
Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see https://quarto.org.
Running Code
When you click the Render button a document will be generated that includes both content and the output of embedded code. You can embed code like this:

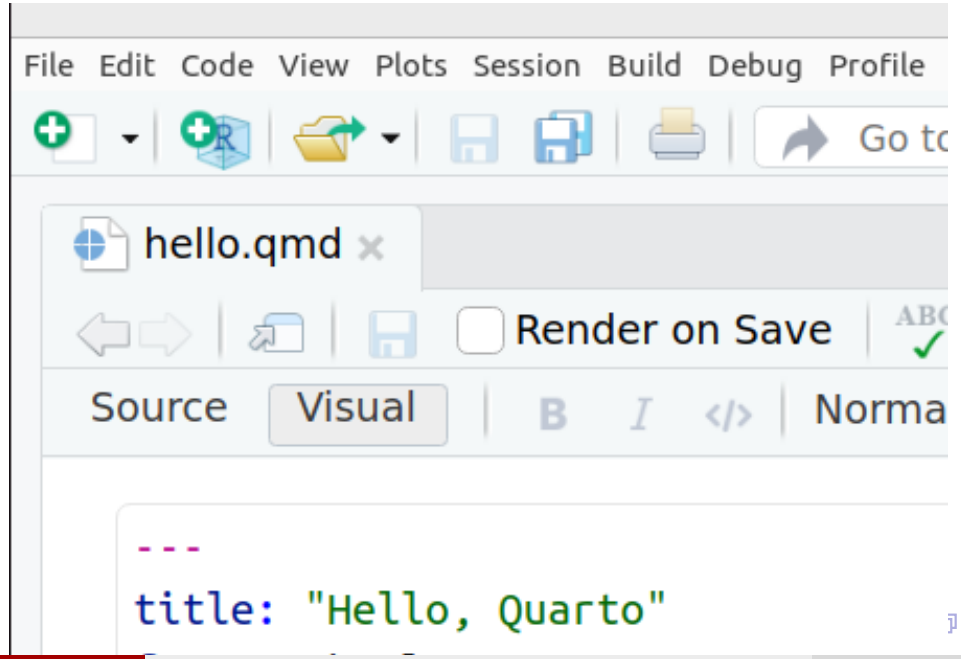
Viewing

- There are a few useful options for viewing which can be accessed by clicking the gear wheel next to the Render button.
- For example, if you select Preview in Window then your HTML file will be previewed in your web browser just like any other webpage.

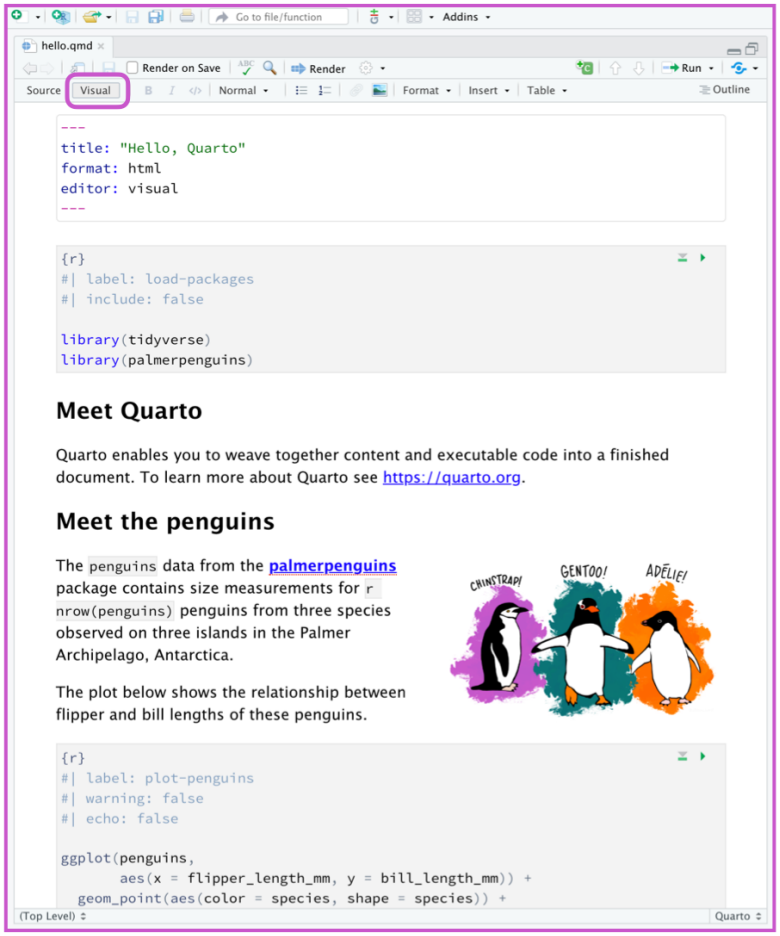


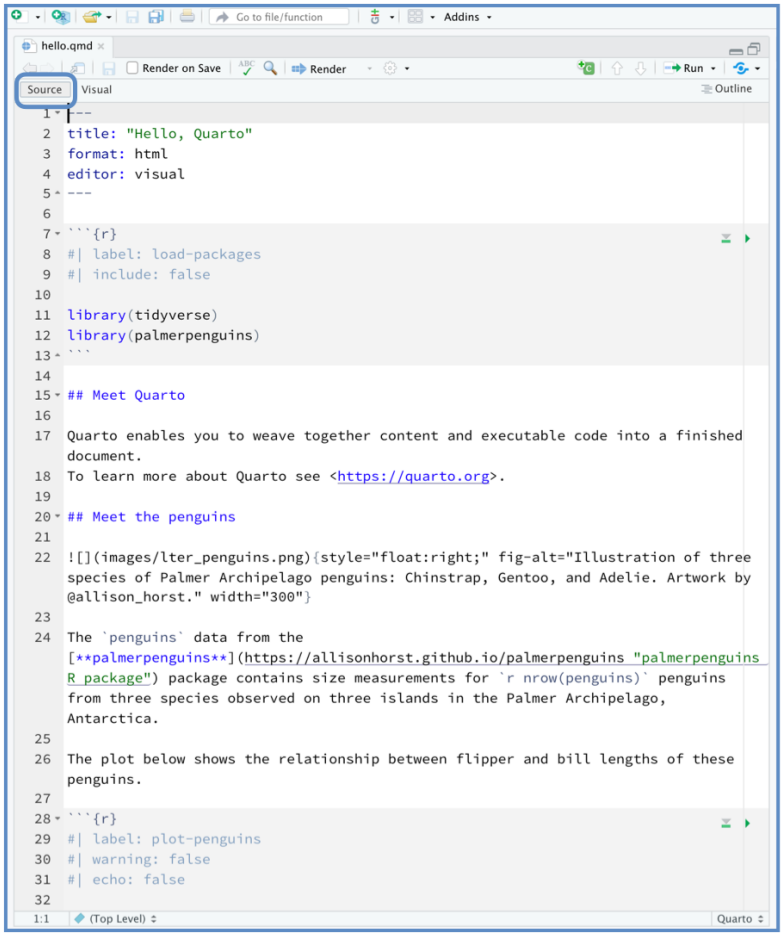
Hello, Quarto

- For the next few minutes we'll follow the tutorial guide which is available at https://quarto.org/docs/get-started/hello/rstudio.html
- If you'd like to follow along then please download the file `hello.qmd` and open it in `Rstudio`.
- The first thing to note is that we can use the Source Editor or the Visual editor depending on our preference.



Source vs. Visual



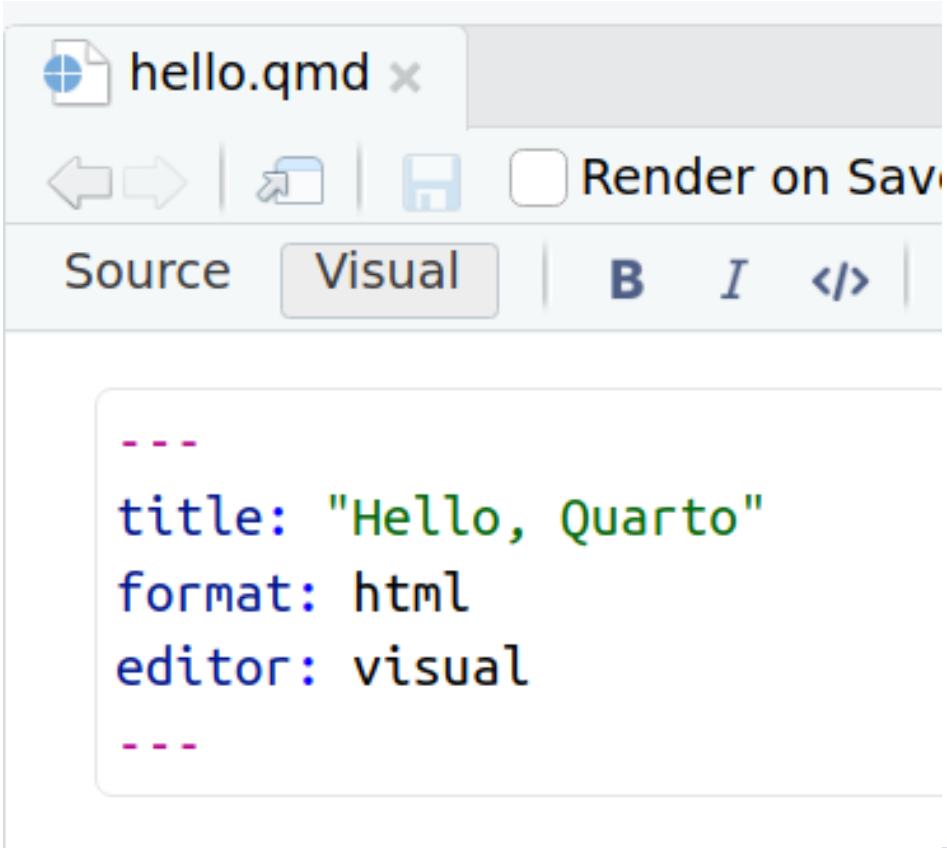


Source vs. Visual

- RStudio's visual editor offers an WYSIWYM authoring experience for markdown.
- For formatting (e.g., bolding text), you can use the toolbar, a keyboard shortcut, or the markdown construct (`**bold**`).
- The plain text source code underlying the document is written for you, and you can view/edit it at any point by switching to source mode for editing.
- You can toggle back and forth between these two modes by clicking on Source and Visual in the editor toolbar.

Quarto File : YAML Header

- YAML: “YAML Ain’t Markup Language”is a human-readable data serialization language. It is commonly used for configuration files and in applications where data is being stored or transmitted.
- The YAML header is contained within $- - -$.



YAML Header

- When rendered, the title, “Hello, Quarto”, will appear at the top of the rendered document with a larger font size than the rest of the document.
- The other two YAML fields denote that the output should be in `html` format and the document should open in the `visual` editor by default.
- The basic syntax of YAML uses pairs in the form `key:value`.
- Other YAML fields commonly found in headers of documents include metadata like `author`, `subtitle`, `date` as well as customization options like `theme`, `fontcolor`, `fig-width`, etc.
- You can find out about all available YAML fields for `HTML` documents at this link https://quarto.org/docs/reference/formats/html.html

Quarto File : Code Chunks

- `R` code chunks identified with `r` with optional chunk options, in YAML style, identified by `#|` at the beginning of the line.

```
{r}
#| label: load-packages
#| include: false

library(tidyverse)
library(palmerpenguins)
```

- In this case, the label of the code chunk is `load-packages`, and we set `include` to `false` to indicate that we don't want the chunk itself or any of its outputs in the rendered documents.
- You can also click the 'play' symbol to run the chunk in the console.

Quarto File : Markdown Text

- Text with formatting, including section headers, hyperlinks, an embedded image, and an inline code chunk.
- Quarto uses markdown syntax for text. Since you don't know markdown this could be be a problem but fear not!
- If using the visual editor, you won't need to learn much markdown syntax for authoring your document, as you can use the menus and shortcuts to add a header, bold text, insert a table, etc.

Code in your Document

- Quarto supports executable code blocks within markdown.
- This allows you to create fully reproducible documents and reports. The code required to produce your output is part of the document itself, and is automatically re-run whenever the document is rendered.
- Following the official tutorial at https://quarto.org/docs/get-started/computations/rstudio.html you can follow along by downloading the file `computations.qmd`

computations.qmd

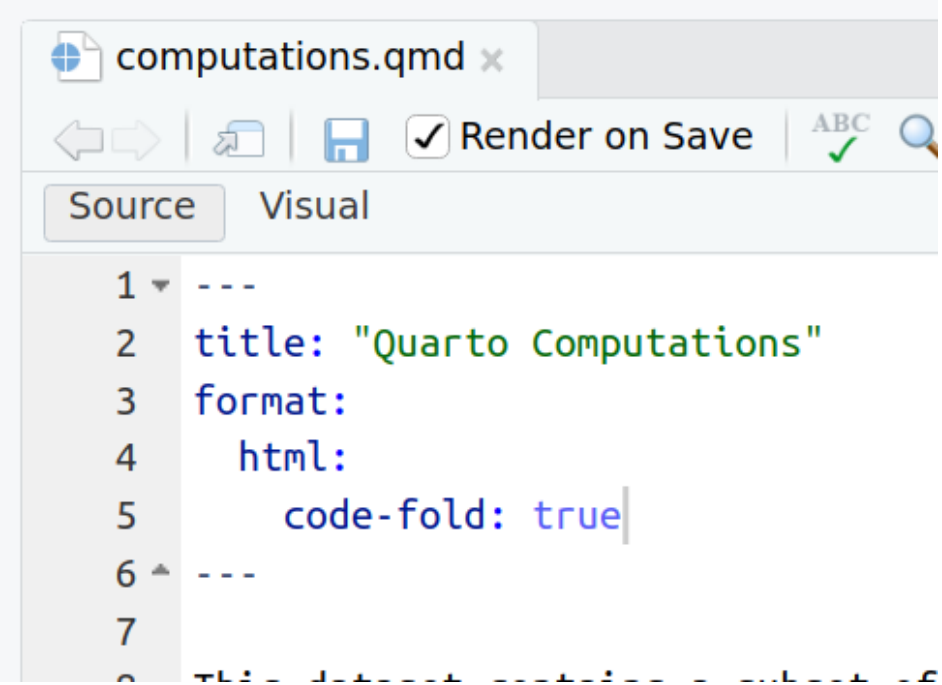
- By default, the code and its output are displayed within the rendered document.
- To hide all of the code and just show the output - specify `echo: false` within the `execute` option in the YAML.
- You might want to selectively enable code `echo` for some cells. To do this add the `echo: true` cell option. Try this with the chunk labelled scatterplot.

computation.qmd

- The echo option can be set to `true`, `false`, or `fenced`.
- There are a large number of other options available for cell output, for example `warning` for showing/hiding warnings (which can be especially helpful for package loading messages), `include` as a catch all for preventing any output (code or results) from being included in output, and `error` to prevent errors in code execution from halting the rendering of the document (and print the error in the rendered document).
- See the Knitr Cell Options documentation for additional details https://quarto.org/docs/reference/cells/cells-knitr.html

computation.qmd

- Rather than hiding code entirely, you might want to fold it and allow readers to view it at their discretion. You can do this via the `code-fold` option. Remove the `echo` option we previously added and add the `code-fold HTML` format option.



computation.qmd

- You can also provide global control over code folding. Try adding `code-tools: true` to the HTML format options.
- Save the document and youâll see that a code menu appears at the top right of the rendered document that provides global control over showing and hiding all code.
- The `code-link` option enables hyper-linking of functions within code blocks to their online documentation. Try adding `code-link: true` to the HTML format options.

Figures

- We can improve the appearance and accessibility of our plot. We can change its aspect ratio by setting `fig-width` and `fig-height`, provide a `fig-cap`, modify its `label` for cross referencing, and add alternative text with `fig-alt`.
- Let's add some options and a cross reference.

```
{r}
#| label: fig-scatterplot
#| fig-cap: "City and highway mileage for 38 popular models of
cars."
#| fig-alt: "Scatterplot of city vs. highway mileage for cars
,where points are colored by the number of cylinders. The plot
displays a positive, linear, and strong relationship between
city and highway mileage, and mileage increases as the number
of cylinders decreases."
#| fig-width: 6
#| fig-height: 3.5
```

Columns

- Let's add another plot to our chunk - a scatterplot where the points are colored by engine displacement, using a different color scale.
- Our goal is to display these plots side-by-side (i.e., in two columns), with a descriptive subcaption for each plot.
- Since this will produce a wider visualization we'll also use the column option to lay it out across the entire page rather than being constrained to the body text column.
- There's a lot going on here so let's do this step by step.

- For code cells with multiple outputs adding the `fig-subcap` option enables us to treat them as subfigures.
- We also added an option to control how multiple figures are laid out-in this case we specified side-by-side in two columns.
- If you have 3, 4, or more figures in a panel there are many options available for customizing their layout. See the article https://quarto.org/docs/authoring/figures.html#figure-panels for details.
- Finally, we added an option to control the span of the page that our figures occupy. This allows our figure display to span out beyond the normal body text column. See the documentation on Article Layout https://quarto.org/docs/authoring/article-layout.html to learn about all of the available layout options.

And much more

https://quarto.org/docs/get-started/authoring/rstudio.html